HIL Simulation Report: dq Current Controller Parametric Sweep

Manuel

2026-06-12

Contents

1 Introduction

This report documents the Python-based Hardware-In-the-Loop (HIL) simulation framework developed to characterise the dynamic response of a dq-frame current controller running on a Typhoon HIL Virtual HIL (VHIL) device.

The primary objective is to evaluate how the closed-loop current bandwidth parameter τ_c — the desired closed-loop time constant of the current controller — affects the transient response to reference steps in both the d-axis (i_{sd}) and q-axis (i_{sq}) currents. Three values are swept:

$$\tau_c \in \{1\,\text{ms},\ 5\,\text{ms},\ 10\,\text{ms}\}$$

The simulation is driven entirely from Python using the `typhoon.api.hil` API, with no physical HIL hardware required. The compiled model (`dq_current_ctrl_v3_vhil.cpd`) runs inside TyphoonSim at approximately 0.1× real time with a 1 µs simulation step.

---

2 Code Architecture

2.1 Module Overview

The codebase is split into three Python modules with clear separation of concerns:

Module	Role
`main.py`	Entry point. Defines all configuration constants, orchestrates the parametric sweep loop, and calls the plotting functions after all runs are complete.
`hil_simulation.py`	HIL abstraction layer. Wraps every `typhoon.api.hil` call into named functions. Contains all simulation lifecycle, capture, SCADA, and contactor logic.
`plotting.py`	Post-processing layer. Reads the CSV result files and generates SVG and PDF figures for individual runs, overlay comparisons, and zoomed views.

2.2 Dependency Graph

```
main.py
  hil_simulation.py      (simulation control)
    typhoon.api.hil      (Typhoon HIL API)
    typhoon.api.schematic_editor  (TSE compilation)
  plotting.py            (figure generation)
    pandas               (CSV reading)
    matplotlib           (plotting)
```

2.3 File and Directory Layout

```
HIL_API/
  main.py                  <- entry point
  hil_simulation.py        <- HIL abstraction layer
  plotting.py              <- figure generation
  docs/
    report.pdf             <- this document
  results/
    step_id_ref_iq_ref_tau_c_0.001.csv
    step_id_ref_iq_ref_tau_c_0.005.csv
    step_id_ref_iq_ref_tau_c_0.01.csv
    fig/
      pdf/                 <- PDF figures
      svg/                 <- SVG figures
```

---

3 Function Reference

3.1 `main.py` — Configuration and Sweep Orchestration

`main.py` contains no functions — it is a top-level script. Its structure is:

Configuration constants:

Constant	Value	Description
`TSE_FILE`	`dq_current_ctrl_v3_vhil.tse`	Schematic source file
`EXE_PATH`	`typhoon_hil.exe` (2026.2)	Typhoon HIL executable
`BASE_DIR`	`HIL_API/`	Project root
`FIG_NAME`	`step_id_ref_iq_ref`	Base name for output files
`SIGNALS`	4 SCADA signals	Signals captured to CSV
`EVENTS`	3 timed events	SCADA inputs fired during the simulation
`TAU_C_VALUES`	`[1e-3, 5e-3, 10e-3]`	Parametric sweep values

Sweep loop (`for tau_c in TAU_C_VALUES`): for each value, loads the model, starts the simulation and capture, sets `_c`, schedules the contactor, runs the event loop, and stops.

Post-loop: kills the Typhoon HIL process, then generates three figures (comparison overlay, zoom at 1.5 s, zoom at 2.0 s).

3.2 `hil_simulation.py` — HIL Abstraction Layer

3.2.1 `launch_control_center(exe_path)`

Checks the Windows task list for a running `typhoon_hil.exe` process. If not found, launches it via `subprocess.Popen`. Avoids duplicate instances when the script is re-run.

3.2.2 `kill_control_center()`

Terminates `typhoon_hil.exe` using `taskkill /F /IM`. Called once after the last `_c` run so the process does not remain open after the sweep.

3.2.3 `compile_if_needed(tse_file)`

Compares the modification timestamps of the TSE schematic and the target CPD binary. Compiles only if the TSE is newer or the CPD is missing, using `typhoon.api.schematic_editor`. Returns the path to the compiled CPD.

3.2.4 `load_model(file_path, vhil_device=True)`

Calls `hil.load_model()` with `offlineMode=False` and `vhil_device=True` to load the compiled model into TyphoonSim. Must be called before any simulation command.

3.2.5 `set_scada_input(name, value)`

Thin wrapper around `hil.set_scada_input_value()`. Used in `main.py` to write `_c` to the SCADA block before the simulation loop begins.

3.2.6 `start_simulation_and_capture(log_file, signals, decimation=50, n_samples=10_000_000)`

Performs three actions in sequence:

1. Starts the simulation (`hil.start_simulation()`).
2. Resets the module-level capture buffer.
3. Starts a waveform capture at `decimation × sim_step = 50 µs` resolution for the listed signals, saving to `log_file`.

Returns `sim_step` so the caller can compute contactor timing.

Capture settings:

Parameter	Value
Decimation	50 (→ 50 µs sample interval)
Channels	4 analog: `is_d_ref`, `is_q_ref`, `id`, `iq`
Samples	10 000 000 (covers >2.5 s at 50 µs)
Trigger	`Forced` (immediate)

3.2.7 `schedule_contactor_close(name, close_at_s, sim_step)`

Computes `execute_at = sim_step × floor(close_at_s / sim_step)` to align the command to the simulation grid, avoiding floating-point misalignment. Sets the contactor to software control mode (open), then schedules closure at `execute_at`.

3.2.8 `run_loop(stop_at_s, events, print_interval_s=0.1)`

Main polling loop. At each iteration:

- Reads `sim_time` from the HIL.

- Fires any pending SCADA event whose simulation time has been reached and whose `requires` dependency has already been triggered.
- Prints a telemetry line (voltage, id, iq, contactor state) every 0.1 simulation seconds.
- Breaks when `sim_time >= stop_at_s`.
- Sleeps 10 ms between iterations once all events are dispatched; no sleep while events are pending (tight loop for accuracy).

3.2.9 `stop_simulation(log_file)`

Stops the waveform capture and simulation. Waits up to 3 s for the Typhoon-written CSV to appear. If the file is missing or empty, falls back to `_write_csv_from_buffer()`.

3.2.10 `_write_csv_from_buffer(log_file)` *(internal)*

Deserialises the in-memory capture buffer `(signal_names, y_data, x_data)` into a pandas DataFrame and saves it as CSV. Guards against transposed `y_data` shapes.

3.3 `plotting.py` — Figure Generation

3.3.1 `setup_output_dirs(base_dir)`

Creates `results/fig/svg/` and `results/fig/pdf/` using `os.makedirs(..., exist_ok=True)`. Returns the `results/` path.

3.3.2 `get_run_paths(results_dir, fig_name, tau_c)`

Builds the three output paths (CSV, SVG, PDF) for a single `_c` run, encoding the value in the filename (e.g., `_tau_c_0.001`).

3.3.3 `plot_dq_currents(log_file, svg_path, pdf_path)`

Single-run figure: two subplots (id and iq) with measured current and reference. Saves both SVG and PDF; catches `PermissionError` if the PDF is locked in a viewer.

3.3.4 `plot_dq_currents_comparison(runs, svg_path, pdf_path)`

Overlay figure for all `_c` runs. Each run is plotted in a distinct colour (matplotlib `tab10`). The reference is drawn once from the first run (identical for all). Legend labels use the form τ_c = value.

3.3.5 `plot_dq_currents_zoom(runs, zoom_time, half_window, svg_path, pdf_path)`

Zoomed version of the comparison figure, restricted to `zoom_time ± half_window` seconds. A vertical dotted line marks the exact step time. Called twice: once at $t = 1.5$ s (i_{sd} step) and once at $t = 2.0$ s (i_{sq} step), each with `half_window = 0.1` s.

---

4 Simulation Event Timeline

The following sequence is common to every `_c` run:

Simulation time	Event
$t = 0.0$ s	Simulation starts. Contactor Plant.S1 open. $i_d^\star = 0$ A, $i_q^\star = 0$ A.
$t = 1.0$ s	Plant.S1 contactor closes — DC bus energised.
$t = 1.1$ s	SCADA input `startAC = 1` — AC converter enabled.
$t = 1.5$ s	SCADA input `i_d_ref = 10.0` A — d-axis current step applied.
$t = 2.0$ s	SCADA input `i_q_ref = -10.0` A — q-axis current step applied.
$t = 2.5$ s	Simulation stopped. Capture flushed to CSV.

---

5 Results and Dynamic Response Analysis

5.1 Captured Signals

Four signals are recorded at 50 µs resolution per run:

Signal name	Description
`SCADA.is_d_ref`	d-axis current reference $i_{sd}^\star$ (A)
`SCADA.is_q_ref`	q-axis current reference $i_{sq}^\star$ (A)
`SCADA.id`	Measured d-axis current i_{sd} (A)
`SCADA.iq`	Measured q-axis current i_{sq} (A)

5.2 Parametric Comparison — Full Run

Figure 1 shows the complete time history of i_{sd} and i_{sq} for all three values of τ_c overlaid on a single plot.

5.3 Zoomed View — d-axis Step ($t = 1.5$ s)

Figure 2 shows the d-axis transient in the window $[1.4, 1.6]$ s.

5.4 Zoomed View — q-axis Step ($t = 2.0$ s)

Figure 3 shows the q-axis transient in the window $[1.9, 2.1]$ s.

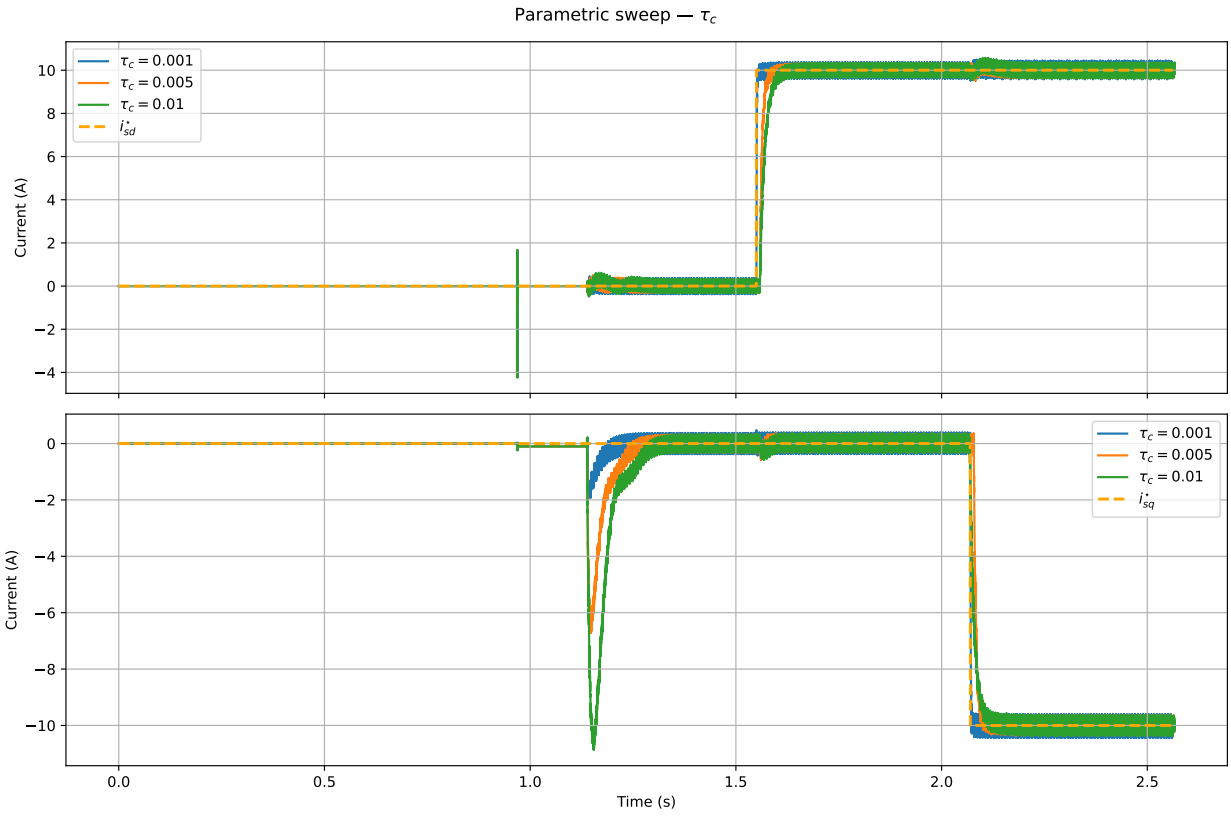

Figure 1: Parametric comparison of i_{sd} and i_{sq} for $\tau_c \in \{1, 5, 10\}$ ms. Reference shown in dashed orange.

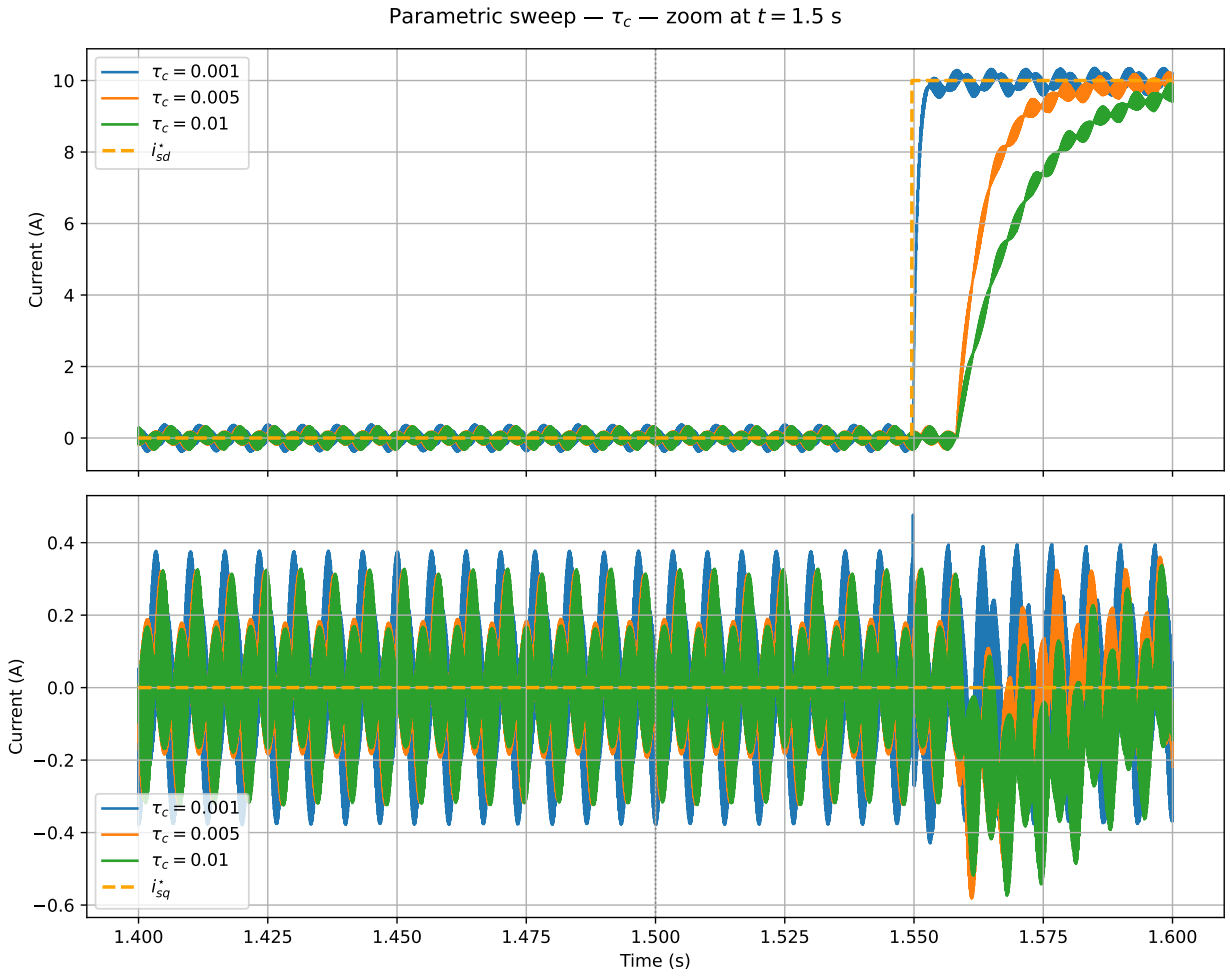

Figure 2: Zoom at $t = 1.5$ s — d-axis step response for all τ_c values.

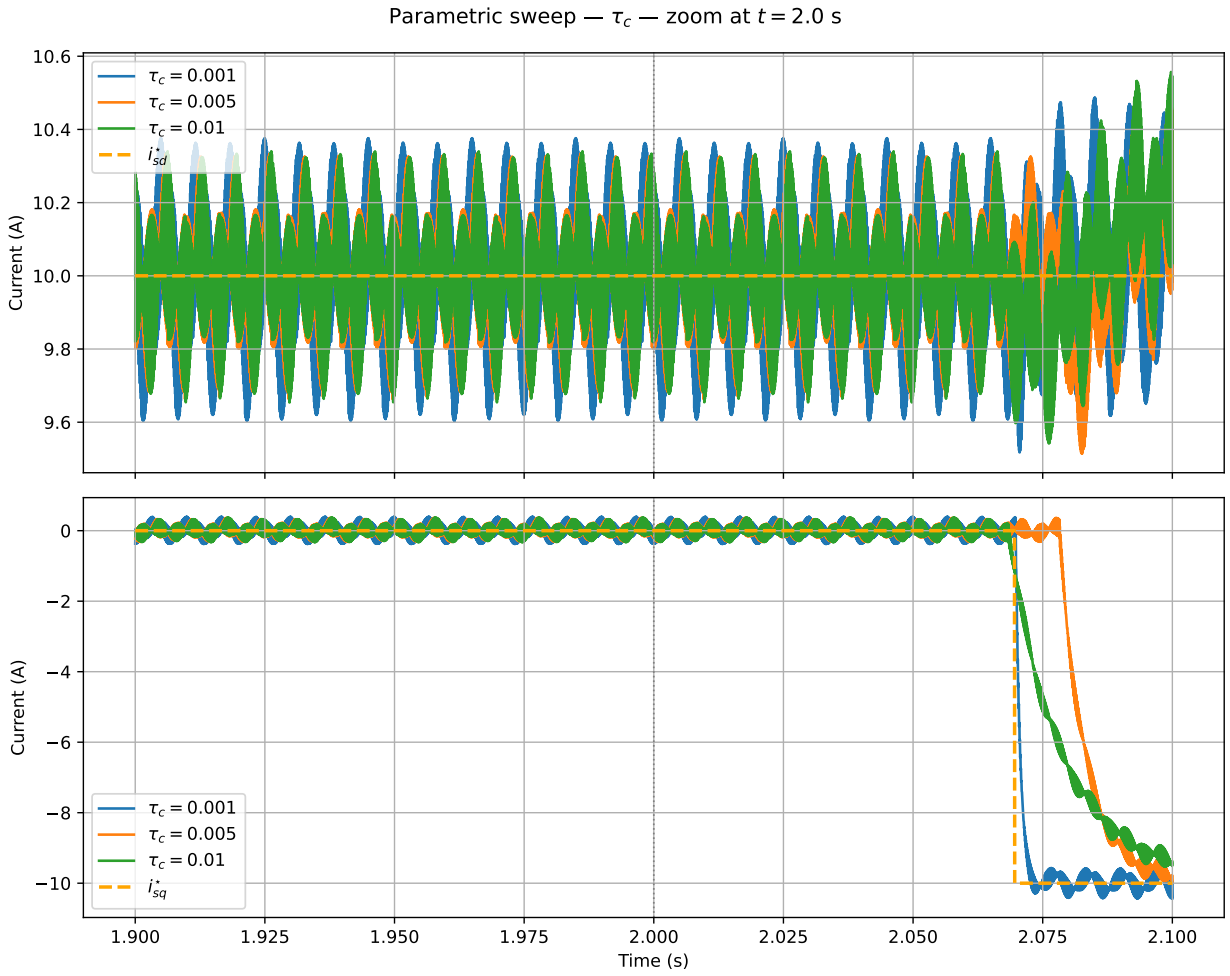

Figure 3: Zoom at $t = 2.0$ s — q-axis step response for all τ_c values.

5.5 Performance Metrics

The following metrics were computed from the CSV data using a 2% settling band:

τ_c (ms)	i_{sd} settle (ms)	i_{sd} overshoot (%)	i_{sq} settle (ms)	i_{sq} overshoot (%)	i_{sd} deviation at i_{sq} step (%)
1	52.5	4.9	72.4	4.5	4.9
5	78.5	5.0	94.2	3.7	5.0
10	98.9	5.8	101.0	3.8	5.8

5.6 Analysis

Bandwidth vs settling time. As τ_c increases from 1 ms to 10 ms, the closed-loop bandwidth decreases and the settling time grows approximately in proportion. The i_{sd} settling time increases from 52.5 ms to 98.9 ms, and the i_{sq} settling time from 72.4 ms to 101.0 ms. This behaviour is expected: τ_c is the desired closed-loop time constant, so a larger value directly corresponds to a slower closed-loop pole.

Overshoot. Peak overshoot remains low across all three cases, ranging from 3.7% to 5.8%. This indicates the current controller is well-damped for the full range of τ_c tested. The slight increase in overshoot for $\tau_c = 10$ ms may be attributed to a lower loop gain that reduces the controller's ability to correct the initial overshoot quickly.

Cross-axis coupling. When the q-axis reference steps at $t = 2.0$ s, the d-axis current i_{sd} is briefly disturbed. The maximum deviation of i_{sd} from its reference tracks τ_c directly: 4.9% for $\tau_c = 1$ ms, 5.0% for $\tau_c = 5$ ms, and 5.8% for $\tau_c = 10$ ms. This is consistent with the fact that slower controllers take longer to reject the cross-coupling disturbance introduced by the dq feed-forward decoupling imperfection.

Asymmetry between axes. In all cases the q-axis settles slightly slower than the d-axis. This asymmetry is typical and reflects differences in the plant parameters (resistance, inductance) and the decoupling terms between the d and q channels.

Summary. For applications requiring fast current dynamics (e.g., direct torque control or active power injection), $\tau_c = 1$ ms offers the best performance with 52.5 ms settling and less than 5% overshoot. For applications where stability margin or parameter robustness is prioritised over speed, $\tau_c = 5$–10 ms provides a more conservative design with acceptable dynamic performance.

---

6 Conclusions

A Python-based HIL parametric sweep framework was developed and validated using the Typhoon HIL VHIL simulation environment. The three-module architecture (`main.py`, `hil_simulation.py`, `plotting.py`) provides clean separation between orchestration, hardware control, and post-processing.

The dq current controller was tested with $\tau_c \in \{1, 5, 10\}$ ms under step references of 10 A (d-axis at $t = 1.5$ s) and -10 A (q-axis at $t = 2.0$ s). Key findings:

- Settling time scales approximately linearly with τ_c, from ~52 ms to ~99 ms on the d-axis.

- Overshoot is contained below 6% for all tested values.
- Cross-axis coupling at the q-step is below 5.8% and decreases with smaller τ_c.
- $\tau_c = 1$ ms represents the best dynamic performance for this model.